

BIG DATA — ARCHITECTURES ET TRAITEMENT DISTRIBUÉ

M2 / Ingénieur — Art Numérique (N5) · Ingénierie Financière (N4)

MODULE 1

Fondations

Données massives et systèmes distribués

TINKU Claude

Année universitaire 2026-2027

Support de cours

Table des matières

1. Qu'est-ce qu'un problème Big Data ?.....	4
1.1 Une question qui est presque toujours mal posée	4
1.2 Une définition opératoire.....	4
1.3 Une méthode en six questions	5
2. Les limites d'une machine unique	6
2.1 Le passage à l'échelle vertical.....	6
2.2 Le mur du scale-up	6
Une limite économique	6
Une limite structurelle.....	6
Une limite matérielle.....	6
2.3 La hiérarchie des latences	7
2.4 Savoir estimer avant de coder.....	7
2.5 Loi d'Amdahl et loi de Gustafson.....	8
3. Caractériser les données	9
3.1 Les « V » : origine, apport et limites.....	9
Critique	9
3.2 Deux axes de classification	10
Selon la structure	10
Selon l'état.....	10
4. Où stocker : entrepôt, lac, lakehouse	11
4.1 L'entrepôt de données	11
4.2 Le lac de données	11
4.3 Le lakehouse	11
4.4 Grille de décision	12
5. Les systèmes distribués	13
5.1 Le passage à l'échelle horizontal	13
5.2 Un modèle de fiabilité	13
Fréquence des pannes.....	13
Probabilité de perte de données.....	13
Les trois réponses à la panne	14
5.3 Partitionner et répliquer.....	14
Trois stratégies de partitionnement.....	15

5.4 Le théorème CAP	15
Démonstration informelle	16
Trois malentendus à éviter	16
5.5 PACELC.....	Erreur ! Signet non défini.
5.6 Localité des données	17
5.7 Le shuffle	17
6. Genèse et état de l'écosystème	19
6.1 Les trois articles de Google.....	19
6.2 De Nutch à Hadoop	19
6.3 Les trois âges de Hadoop.....	20
6.4 Ce qui n'est plus à apprendre	20
6.5 Ce qui compte aujourd'hui	21
6.6 Hadoop est-il mort ?.....	21
7. Exercices d'auto-évaluation	22
Exercice 1 — Qualification.....	22
Exercice 2 — Estimation	22
Exercice 3 — Fiabilité	22
Exercice 4 — CAP et PACELC	23
8. Corrigés des exercices	24
Corrigé de l'exercice 1	24
Corrigé de l'exercice 2	24
Corrigé de l'exercice 3	24
Corrigé de l'exercice 4	25
9. Glossaire du module.....	26

1. Qu'est-ce qu'un problème Big Data ?

1.1 Une question qui est presque toujours mal posée

Demandez à dix ingénieurs à partir de quel volume on parle de Big Data, vous obtiendrez dix réponses différentes, comprises entre le téraoctet et le pétaoctet. Toutes seront défendables, et toutes seront fausses — non pas parce que le chiffre est mal choisi, mais parce que la question elle-même est mal posée.

Deux contre-exemples suffisent à s'en convaincre.

- **Beaucoup de données, aucun problème.** Un studio d'animation conserve 40 To de rushes vidéo. Il les lit séquentiellement, fichier par fichier, quelques fois par mois. Un simple serveur de fichiers avec de gros disques suffit. Il n'y a ici aucun problème de Big Data, seulement un problème de capacité de stockage.
- **Peu de données, un vrai problème.** Une salle de marché reçoit 80 Go de cotations par jour et doit, pour chaque ordre entrant, le rapprocher de l'historique des trente dernières minutes, de trois référentiels et d'un modèle de risque — le tout en moins de cinquante millisecondes. Le volume est modeste ; la contrainte de latence sous charge, elle, impose une architecture distribuée.

Ce que ces deux cas révèlent, c'est que le volume n'est qu'un paramètre parmi plusieurs, et rarement le plus contraignant. Ce qui décide, c'est le **rapport entre le traitement souhaité et la capacité de la machine dont on dispose**.

1.2 Une définition opératoire

DÉFINITION

Un problème relève du **Big Data** lorsque le traitement souhaité ne tient pas dans les ressources d'une seule machine — en volume, en débit ou en délai — et que la solution retenue consiste à répartir les données et le calcul sur plusieurs machines coordonnées.

Cette définition a trois propriétés qui la rendent utilisable, là où les définitions usuelles ne le sont pas.

- **Elle est relative.** Elle dépend explicitement de la machine disponible. La même charge peut être un problème distribué sur un poste de travail et une formalité sur un serveur à 2 To de RAM. C'est une propriété, pas un défaut : elle reflète la réalité de l'ingénierie.
- **Elle est opératoire.** Elle se vérifie par un calcul — mesurer le volume, le débit disponible, le délai exigé — et non par une opinion. Deux ingénieurs partant des mêmes chiffres aboutiront à la même conclusion.
- **Elle nomme la solution autant que le problème.** Elle dit que faire du Big Data, c'est faire du calcul distribué. Tout le contenu de ce cours en découle.

1.3 Une méthode en six questions

Avant toute discussion technologique, une charge de travail se caractérise par six grandeurs mesurables. Cette grille remplace avantageusement l'énumération des « V » que nous critiquons à la section 3.

Question	Grandeur	Décision d'architecture qu'elle oriente
Combien de données au total, et à quel rythme croissent-elles ?	Volume, taux de croissance annuel	Dimensionnement du stockage, choix HDFS / objet
Quel débit d'arrivée en régime nominal, et en pic ?	Mio/s, événements/s	Dimensionnement de l'ingestion, partitionnement Kafka
Quel délai maximal entre l'événement et la décision ?	Latence de bout en bout	Traitement par lots ou en continu
Quelle proportion des données une requête typique lit-elle ?	Sélectivité	Format ligne ou colonne, partitionnement, index
Quel est le ratio lecture / écriture, et les données sont-elles modifiées ?	Ratio, taux de mise à jour	Format de fichier immuable ou format de table transactionnel
Quelle décision cette donnée permet-elle de prendre ?	Valeur métier	Justifie — ou annule — le projet

Grille de qualification d'une charge de travail. Elle est appliquée en TD1.

À RETENIR

La dernière question est la seule qui puisse **arrêter** un projet. Si personne dans l'organisation ne sait dire quelle décision sera prise différemment grâce à ces données, il n'y a pas de projet de données — il y a une dépense.

2. Les limites d'une machine unique

2.1 Le passage à l'échelle vertical

Face à une charge qui croît, le premier réflexe — et le bon, dans la majorité des cas — consiste à remplacer la machine par une plus puissante. On parle de *scale-up*, ou de passage à l'échelle vertical. Son mérite principal est ce qu'il **n'impose pas** : ni réécriture du code, ni réseau à administrer, ni gestion de pannes partielles, ni raisonnement sur la cohérence des données.

Il faut mesurer jusqu'où cette voie mène aujourd'hui. Une instance AWS `u-24tb1.metal` offre 448 processeurs logiques et **24 tébi octets de mémoire vive**. La quasi-totalité des jeux de données transactionnels d'entreprise y tiennent intégralement en mémoire. Des moteurs mono-machine modernes comme DuckDB ou Polars exploitent cette réalité et ont récupéré une large part des usages qui allaient par défaut vers Spark il y a dix ans.

ATTENTION

Ne distribuez jamais un traitement qui tient sur une machine. Un cluster ajoute un coût matériel, mais surtout un coût humain permanent : administration, supervision, diagnostic de pannes partielles, montées de version. De nombreuses équipes ont réduit leurs coûts d'un ordre de grandeur en revenant d'une chaîne Spark à un traitement mono-machine, parce que leur volume réel se comptait en dizaines de gigaoctets.

2.2 Le mur du scale-up

Cette voie finit néanmoins par buter sur trois familles de limites.

Une limite économique

La relation entre le prix d'un serveur et ses performances n'est pas linéaire. Doubler la capacité mémoire d'une machine haut de gamme coûte considérablement plus que doubler son prix, parce qu'on quitte le marché de série pour celui des composants spécialisés. Il existe une machine maximale, et le coût par unité de capacité y est le plus élevé du marché.

Une limite structurelle

- **Point unique de défaillance.** Toute la charge repose sur une machine. Sa panne arrête le service, et sa réparation prend des heures.
- **Indisponibilité pour maintenance.** Une mise à jour du système d'exploitation impose un arrêt planifié.
- **Élasticité nulle.** On ne loue pas la moitié d'un serveur pour trois heures. La capacité est dimensionnée pour le pic et gaspillée le reste du temps.

Une limite matérielle

La fréquence des processeurs stagne depuis le milieu des années 2000 : les gains de performance viennent désormais de la multiplication des cœurs, c'est-à-dire du parallélisme. Or exploiter cent cœurs dans une machine pose déjà des problèmes de synchronisation très proches de ceux du calcul distribué — sans en offrir la tolérance aux pannes.

À RETENIR

Le passage à l'échelle horizontale ne rend pas un traitement plus rapide. Il permet d'aller **plus loin** en capacité totale, et de **survivre aux pannes**. Ce sont deux bénéfices différents de la vitesse, et il faut les nommer correctement.

2.3 La hiérarchie des latences

Toute l'architecture des systèmes distribués découle d'un fait matériel simple : les temps d'accès aux différents niveaux de mémoire s'échelonnent sur huit ordres de grandeur. Un ingénieur data doit connaître ce tableau de mémoire.

Opération	Latence	Échelle humaine ($\times 10^9$)
Accès au cache L1	≈ 1 ns	1 seconde
Accès au cache L2	≈ 4 ns	4 secondes
Accès à la mémoire vive	≈ 100 ns	1 minute 40
Lecture de 1 Mio en mémoire vive	≈ 50 μ s	14 heures
Lecture aléatoire sur SSD NVMe	≈ 100 μ s	1 jour
Lecture de 1 Mio sur SSD NVMe	≈ 200 μ s	2 jours
Aller-retour réseau, même centre de données	≈ 500 μ s	6 jours
Déplacement de tête d'un disque dur	≈ 5 ms	2 mois
Lecture de 1 Mio sur disque dur	≈ 5 ms	2 mois
Aller-retour Paris $\rightarrow$ Californie $\rightarrow$ Paris	≈ 150 ms	5 ans

Ordres de grandeur, d'après les « Latency Numbers Every Programmer Should Know » de Jeff Dean. Ce sont des ordres de grandeur, pas des mesures.

La colonne de droite est l'outil pédagogique : en multipliant toutes les durées par un milliard, on les ramène à une échelle perceptible. Un accès mémoire dure alors une minute quarante ; un aller-retour réseau, six jours ; une lecture sur disque dur, deux mois.

À RETENIR

Entre la mémoire vive et le réseau local, il y a un facteur d'environ **5 000**. Entre la mémoire vive et le disque dur, un facteur d'environ **50 000**. Chaque fois que ce cours recommandera d'« éviter un shuffle », de « filtrer au plus tôt » ou de « diffuser la petite table », c'est de ces deux facteurs qu'il s'agira.

2.4 Savoir estimer avant de coder

La compétence la plus utile de ce cours n'est pas de savoir écrire du Spark : c'est de savoir estimer, au facteur dix près et sans machine, combien de temps un traitement devrait prendre. Cela permet de détecter qu'un job qui met six heures aurait dû en mettre vingt minutes — donc qu'il y a un problème à chercher.

Combien de temps pour parcourir 1 Tio ?

```
# Disque dur, lecture séquentielle a 200 Mio/s
  1 Tio = 1 048 576 Mio
  t = 1 048 576 / 200 = 5 243 s ~ 1 h 27

# SSD NVMe a 3 Gio/s
  t = 1 048 576 / 3 072 = 341 s ~ 5 min 41

# 100 disques durs en parallele (le cluster)
  t = 5 243 / 100 = 52 s

# ... mais si le resultat doit transiter par un reseau 10 Gb/s :
  10 Gb/s = 1,25 Go/s = 1 192 Mio/s (des BITS, et du decimal)
  t = 1 048 576 / 1 192 = 880 s ~ 14 min 40
```

Trois enseignements : le parallélisme sur les disques est le gain le plus spectaculaire ; ce gain s'évapore si l'on retransfère tout par le réseau ; d'où le principe de localité (section 5.6).

2.5 Loi d'Amdahl et loi de Gustafson

Le parallélisme a une borne théorique qu'il est indispensable de connaître, ne serait-ce que pour ne pas attendre d'un cluster ce qu'il ne peut pas donner.

DÉFINITION

Loi d'Amdahl (1967). Si une proportion s du temps d'exécution d'un programme est intrinsèquement séquentielle et ne peut être parallélisée, l'accélération obtenue avec N unités de calcul vaut au plus :

- $S(N) = 1 / (s + (1 - s) / N)$
- et lorsque N tend vers l'infini, $S(N)$ tend vers $1 / s$.

Les conséquences numériques sont brutales. Pour $s = 5\%$:

Nombre de machines N	Accélération $S(N)$	Efficacité $S(N)/N$
1	$\times 1,0$	100 %
10	$\times 6,9$	69 %
100	$\times 16,8$	17 %
1 000	$\times 19,6$	2 %
10 000	$\times 19,96$	0,2 %

Avec seulement 5 % de code séquentiel, passer de 100 à 10 000 machines n'apporte que 19 % d'accélération supplémentaire.

En pratique, la fraction séquentielle s n'est pas seulement du code : elle inclut la planification du job, la lecture des métadonnées, la collecte du résultat sur le pilote, l'écriture finale. C'est pourquoi un `collect()` mal placé dans un programme Spark suffit à annuler le bénéfice de cent machines — nous le vérifierons au module 3.

NUANCE

Loi de Gustafson (1988). Amdahl suppose la taille du problème constante. Or dans le monde réel, on n'ajoute pas des machines pour traiter le même jeu de données plus vite : on en ajoute pour traiter un jeu de données plus grand dans le même temps. Sous cette hypothèse, la part séquentielle devient proportionnellement négligeable et l'accélération redevient quasi linéaire. En résumé : **Amdahl borne la latence, Gustafson décrit le débit.** Le Big Data est presque toujours un problème de débit.

3. Caractériser les données

3.1 Les « V » : origine, apport et limites

En 2001, Doug Laney, analyste au META Group — cabinet racheté depuis par Gartner — publie une note intitulée *3D Data Management : Controlling Data Volume, Velocity and Variety*. Il y décrit les difficultés nouvelles que le commerce en ligne naissant posait à la gestion des données.

ATTENTION

Contrairement à ce qu'affirment de très nombreux supports, **Doug Laney n'a pas inventé l'expression « Big Data »**. Le terme circulait déjà dans les communautés scientifiques — astronomie, physique des particules — dans les années 1990. La récupération du triptyque de Laney comme définition du Big Data est postérieure d'une dizaine d'années et lui est extérieure.

Le triptyque a néanmoins un mérite réel : il rappelle que **le volume n'est qu'une dimension parmi plusieurs**, et il fournit trois axes de questionnement en début de projet.

V	Question qu'il pose	Ce qu'il oriente
Volume	Combien de données, et à quel rythme croissent-elles ?	Le dimensionnement du stockage
Vélocité	Quel délai maximal entre l'événement et la décision ?	Le choix lots / flux continu
Variété	Combien de sources, quels schémas, stables ou non ?	Le format de fichier et le catalogue
Véracité	Quelle confiance dans la donnée, quel taux d'erreur ?	Le budget consacré à la qualité
Valeur	Quelle décision cette donnée permet-elle de prendre ?	L'existence même du projet
Variabilité	La qualité et le débit varient-ils dans le temps ?	Le dimensionnement des pics

Critique

À un niveau M2, il faut aller plus loin que la récitation. Le modèle des V présente quatre faiblesses.

1. **Il n'est pas discriminant.** Toute donnée, sans exception, possède un volume, une vitesse d'arrivée et un format. Le modèle ne permet donc de trancher aucun cas limite.
2. **Il n'est pas mesurable.** Aucun seuil, aucune unité, aucun protocole. Deux ingénieurs compétents peuvent qualifier différemment le même système sans qu'aucun ait tort.
3. **Il est extensible à l'infini.** La littérature commerciale va jusqu'à dix ou dix-sept V. Cette inflation est le signe d'un dispositif rhétorique, non d'un modèle scientifique : un modèle se réfute, une liste s'allonge.
4. **Il omet la dimension la plus déterminante**, à savoir la **structure des accès**. Lit-on l'intégralité du jeu de données ou une ligne isolée ? Écrit-on une fois pour toutes ou met-on à jour en permanence ? C'est cette question, et non les V, qui décide de l'architecture.

Ce dernier point mérite un développement. Trois patrons d'accès mènent à trois architectures radicalement différentes :

- **Lecture séquentielle massive, écriture en ajout** — on parcourt des milliards de lignes pour en agréger quelques colonnes. Réponse : format colonne (Parquet), stockage objet ou HDFS, traitement par lots distribué.
- **Lecture ponctuelle par clé, à très faible latence** — on veut une ligne précise en quelques millisecondes. Réponse : base NoSQL clé-valeur ou orientée colonnes (HBase, Cassandra, DynamoDB). Un moteur analytique distribué serait ici absurde.
- **Mise à jour fréquente ligne à ligne avec garanties transactionnelles** — réponse : base relationnelle, ou format de table transactionnel de type Hudi ou Iceberg.

À RETENIR

Utilisez les V comme une **liste de questions à poser**, jamais comme une définition. Pour caractériser réellement une charge, employez la grille en six questions de la section 1.3 et déterminez le patron d'accès dominant.

3.2 Deux axes de classification

Les données se classent selon deux axes indépendants, qu'il ne faut pas confondre.

Selon la structure

Catégorie	Caractéristique	Exemples — finance	Exemples — art numérique
Structurées	Schéma fixe, connu à l'écriture	Écritures comptables, cours de clôture	Catalogue d'œuvres, table des utilisateurs
Semi-structurées	Schéma souple, porté par la donnée elle-même	Messages SWIFT, flux de cotations JSON	Événements d'interface, métadonnées EXIF
Non structurées	Aucun schéma directement exploitable	Rapports d'analystes, courriers, appels	Images, vidéos, sons, modèles 3D

Selon l'état

Les données **au repos** (*at rest*) sont stockées et interrogées a posteriori ; on les traite par lots, avec des délais de l'ordre de la minute à l'heure. Les données **en mouvement** (*in motion*) forment un flux continu d'événements ; on les traite en continu, avec des délais de l'ordre de la milliseconde à la seconde.

Ces deux états ne désignent pas deux types de données mais **deux moments de la vie de la même donnée**. Une transaction bancaire est d'abord un événement dans un flux, puis une ligne dans un historique. L'architecture *Kappa*, étudiée au module 5, tire les conséquences de cette continuité en n'implémentant qu'un seul chemin de traitement, là où l'architecture *Lambda* historique en maintenait deux.

4. Où stocker : entrepôt, lac, lakehouse

4.1 L'entrepôt de données

L'entrepôt de données (*data warehouse*), formalisé dans les années 1990 par Bill Inmon et Ralph Kimball, repose sur le principe du **schema-on-write** : la donnée est nettoyée, conformée et modélisée **avant** d'être stockée. Le schéma est un contrat imposé à l'écriture.

- **Forces** — requêtes rapides et prévisibles, qualité garantie par construction, transactions ACID, gouvernance et traçabilité solides, SQL universellement maîtrisé.
- **Faiblesses** — coût de stockage élevé, rigidité du modèle, cycle de développement long, et surtout : tout ce qui n'entre pas dans le schéma est refusé, ce qui exclut de fait les données non structurées.

4.2 Le lac de données

Le lac de données (*data lake*), popularisé dans les années 2010 avec Hadoop puis avec le stockage objet, inverse le principe : **schema-on-read**. On stocke la donnée brute telle qu'elle arrive, et l'on n'interprète sa structure qu'au moment de la lecture.

- **Forces** — accepte tout format, coût de stockage très faible, aucune transformation à l'ingestion, ne préjuge pas des usages futurs, indispensable pour l'apprentissage automatique qui a besoin du signal brut.
- **Faiblesses** — aucune transaction, aucune garantie de cohérence entre lecteurs et écrivains, qualité non contrôlée, schémas qui dérivent silencieusement.

ATTENTION

Le marécage de données (*data swamp*). Un lac dépourvu de catalogue, de propriétaire identifié et de documentation devient rapidement inexploitable : on y a tout déversé, et plus personne ne sait ce qu'il contient ni ce qui est fiable. Ce n'est pas une caricature : c'est le destin d'une large part des lacs construits entre 2012 et 2018, et l'une des raisons du discrédit passager de Hadoop.

4.3 Le lakehouse

Le lakehouse, formalisé au début des années 2020, cherche à conserver le coût et la souplesse du lac tout en récupérant les garanties de l'entrepôt. Sa contribution décisive est l'introduction d'une **couche de format de table** — Apache Iceberg, Delta Lake, Apache Hudi — posée entre les fichiers et les moteurs de calcul.

Techniquement, ces formats maintiennent, à côté des fichiers de données, un **journal de métadonnées** qui recense quels fichiers composent la table à un instant donné. Une écriture ne modifie jamais un fichier existant : elle en ajoute de nouveaux, puis publie atomiquement une nouvelle version du journal. De ce mécanisme découlent directement :

- **Les transactions ACID** — un lecteur voit soit l'ancienne version, soit la nouvelle, jamais un état intermédiaire ;
- **Le voyage dans le temps** (*time travel*) — il suffit de lire une version antérieure du journal pour interroger la table telle qu'elle était hier ;
- **L'évolution de schéma** — ajouter ou renommer une colonne devient une opération de métadonnées, sans réécrire les données ;
- **L'élitage de fichiers** — le journal porte des statistiques par fichier, ce qui permet d'ignorer d'emblée ceux qui ne peuvent pas contenir de ligne pertinente.

L'architecture cible, celle que ce cours construit module après module, s'empile ainsi :

Couche	Rôle	Technologies	Module
Consommation	Restitution et modélisation	SQL, tableaux de bord, notebooks, MLlib	M6
Moteurs	Calcul distribué sur les mêmes fichiers	Spark, Trino, Flink	M3, M4, M5
Format de table	ACID, versions, évolution de schéma	Iceberg, Delta, Hudi	M2 (S5)
Format de fichier	Stockage colonne compressé	Parquet, ORC	M2 (S4)
Stockage	Persistance brute, peu coûteuse	S3, MinIO, HDFS	M2 (S3, S5)

4.4 Grille de décision

Critère	Data Warehouse	Data Lake	Lakehouse
Moment du schéma	Écriture	Lecture	Écriture souple
Transactions ACID	Oui	Non	Oui
Coût de stockage	Élevé	Très faible	Très faible
Données non structurées	Non	Oui	Oui
Gouvernance	Forte	Faible	Moyenne à forte
Voyage dans le temps	Limité	Non	Oui
Adapté à l'apprentissage	Mal	Oui	Oui
Risque principal	Rigidité, coût	Marécage	Complexité d'exploitation

NUANCE

Le lakehouse n'est pas gratuit. Il exige un catalogue, une politique de compaction des petits fichiers, une gestion de l'expiration des instantanés et une équipe capable de l'exploiter. Une équipe de trois personnes gérant 200 Gio de données n'en a très probablement pas besoin : un entrepôt géré ou même une base relationnelle bien conçue sera plus rapide à mettre en œuvre et moins coûteuse à maintenir.

5. Les systèmes distribués

5.1 Le passage à l'échelle horizontal

Le *scale-out* substitue à une machine exceptionnelle un grand nombre de machines banales. La capacité totale devient la somme des capacités, la croissance est linéaire et sans plafond théorique, et l'on peut ajouter ou retirer des nœuds sans interrompre le service.

ATTENTION

L'expression *commodity hardware*, souvent traduite par « matériel bon marché », induit en erreur. Elle ne désigne pas du matériel de récupération, mais du **matériel de série** : des serveurs standards, sans composant propriétaire, interchangeables et disponibles chez n'importe quel fournisseur. Un nœud de cluster typique en 2026 comporte 32 à 64 cœurs, 128 à 512 Gio de mémoire et une dizaine de disques.

La contrepartie est entièrement logicielle : réseau, pannes partielles, cohérence, coordination. C'est le sujet de tout le reste de ce cours.

5.2 Un modèle de fiabilité

Sur un grand cluster, la panne cesse d'être un incident pour devenir un régime de fonctionnement normal. Ce basculement se démontre par un calcul élémentaire.

Fréquence des pannes

Soit un cluster de N serveurs, chacun ayant un temps moyen entre pannes (MTBF) de trois ans, soit 1 095 jours. En supposant les pannes indépendantes et de taux constant, le nombre attendu de pannes par jour vaut $N / 1\,095$.

Taille du cluster N	Pannes attendues par jour	Intervalle moyen entre pannes
1	0,0009	3 ans
100	0,09	11 jours
1 000	0,91	26 heures
10 000	9,1	2 h 40

Au-delà de quelques centaines de nœuds, il devient certain qu'une machine est en panne à tout instant. Le système doit donc absorber cet événement **sans intervention humaine et sans interruption de service**. Et ce modèle ne compte que les pannes matérielles : s'y ajoutent, plus fréquentes encore, les partitions réseau, les disques saturés, les processus figés et les mises à jour ratées.

Probabilité de perte de données

La réplication ne supprime pas le risque de perte : elle le rend improbable. On peut le quantifier. Supposons un cluster de N nœuds dont k tombent simultanément, et un bloc répliqué r fois sur des nœuds tirés uniformément. Le bloc est perdu si et seulement si ses r répliques se trouvent tous parmi les k nœuds défaillants :

$$P(\text{bloc perdu}) = C(k, r) / C(N, r)$$

Application : $N = 1000$ noeuds, $r = 3$ replicas

$k = 3$ noeuds tombent simultanément
 $P = C(3,3) / C(1000,3) = 1 / 166\,167\,000 \sim 6,0e-9$

$k = 10$ noeuds tombent simultanément
 $P = C(10,3) / C(1000,3) = 120 / 166\,167\,000 \sim 7,2e-7$

$k = 50$ noeuds (panne d'un rack entier, par exemple)
 $P = C(50,3) / C(1000,3) = 19\,600 / 166\,167\,000 \sim 1,2e-4$

Avec 100 000 blocs dans le cluster, le dernier cas donne une espérance d'environ 12 blocs perdus. D'où l'importance du placement des réplicas.

À RETENIR

Ce calcul suppose un placement **uniforme et indépendant** des réplicas. Or les pannes réelles ne sont pas indépendantes : une alimentation ou un commutateur défaillant fait tomber un rack entier d'un coup. C'est précisément pourquoi HDFS met en œuvre le *rack awareness* — il impose qu'au moins un réplica se trouve dans un autre rack, ce qui rend la panne d'un rack entier survivable par construction. Nous détaillerons ce mécanisme au module 2.

Les trois réponses à la panne

Tout système distribué, sans exception, met en œuvre les trois mêmes mécanismes.

Mécanisme	Principe	Dans HDFS	Dans Spark
Répliquer	Chaque donnée existe en n copies sur des machines distinctes	Réplication $\times 3$ par bloc, ou erasure coding	Aucune par défaut : le lignage remplace la copie
Détecter	Battements de cœur périodiques ; le silence vaut décès	Heartbeat DataNode toutes les 3 s	Heartbeat de l'executor vers le driver
Reprendre	Le travail perdu est relancé ailleurs à partir d'un état connu	Re-réplication automatique des blocs sous-répliqués	Re-calcul des seules partitions perdues

La colonne Spark mérite un commentaire. Plutôt que de répliquer les données intermédiaires, Spark mémorise la **suite d'opérations qui les a produites** — c'est le *lignage* (*lineage*). Lorsqu'une partition est perdue, il suffit de rejouer cette suite d'opérations pour la reconstruire. Cette idée, publiée en 2012 dans l'article fondateur sur les RDD, économise l'essentiel du coût mémoire de la tolérance aux pannes. Elle est développée au module 3.

5.3 Partitionner et répliquer

Ces deux opérations sont fréquemment confondues alors qu'elles répondent à des besoins distincts et se cumulent. HDFS, par exemple, découpe d'abord un fichier en blocs (partitionnement), **puis** réplique chaque bloc.

	Partitionnement (sharding)	Réplication
Objectif	Capacité et parallélisme	Disponibilité et localité
Principe	Découper en morceaux disjoints	Dupliquer chaque morceau n fois
Permet	De dépasser la taille d'une machine ; de traiter en parallèle	De survivre à la panne ; de lire depuis plusieurs endroits
Coût	Complexité du routage ; risque de déséquilibre	Espace disque ; maintien de la cohérence entre copies
Pathologie	Le déséquilibre (<i>skew</i>)	La divergence entre réplicas

Trois stratégies de partitionnement

Stratégie	Principe	Avantage	Piège
Par plage	Clés triées découpées en intervalles	Requêtes par intervalle très efficaces	Points chauds si les clés sont chronologiques
Par hachage	$\text{partition} = \text{hash}(\text{clé}) \bmod N$	Répartition statistiquement uniforme	Perd l'ordre ; rééquilibrage coûteux quand N change
Par liste	Une table associe explicitement clé et nœud	Contrôle total, rééquilibrage souple	Le répertoire devient un composant critique

EXEMPLE

Le piège du partitionnement par plage sur clé chronologique concerne directement la filière Ingénierie Financière. Si l'on partitionne des transactions par date, **toutes** les écritures du jour courant visent la même partition : un nœud sature pendant que les autres sont oisifs. Le remède classique consiste à composer la clé — par exemple `hash(id_compte)` suivi de la date — ou à *saler* la clé, technique que nous appliquerons au module 4 pour corriger les jointures déséquilibrées.

5.4 Le théorème CAP

Formulé comme conjecture par Eric Brewer en 2000 et démontré par Seth Gilbert et Nancy Lynch en 2002, le théorème CAP est le résultat le plus cité — et le plus mal cité — de l'informatique distribuée.

DÉFINITION

Théorème CAP. Un système distribué soumis à une partition réseau doit choisir entre rester cohérent et rester disponible. Il ne peut pas garantir les deux simultanément.

- **Cohérence (C)** — toute lecture renvoie la dernière écriture validée, ou bien une erreur.
- **Disponibilité (A)** — toute requête reçoit une réponse non erronée, sans garantie sur sa fraîcheur.
- **Tolérance au partitionnement (P)** — le système continue de fonctionner malgré la perte arbitraire de messages entre nœuds.

Démonstration informelle

Considérons deux nœuds A et B détenant chacun un réplica d'une même valeur $x = 0$, et supposons le lien entre eux rompu. Un client écrit $x = 1$ sur A. Un second client lit x sur B. Deux comportements seulement sont possibles :

- B répond 0. Le système est **disponible** mais **incohérent** : il a renvoyé une valeur périmée.
- B refuse de répondre, ou renvoie une erreur, en attendant de pouvoir se synchroniser avec A. Le système est **cohérent** mais **indisponible**.

Aucune troisième voie n'existe, puisque par hypothèse aucun message ne peut circuler entre A et B. Le théorème est donc, sous cette forme, presque évident — ce qui n'enlève rien à son utilité comme cadre de raisonnement.

Trois malentendus à éviter

5. « **Il faut choisir deux propriétés sur trois.** » Faux. La tolérance au partitionnement n'est pas un choix d'architecture : les partitions réseau **surviennent**, que le concepteur le veuille ou non. Un système distribué doit donc toujours tolérer P. Le seul choix réel est binaire — C ou A — et il ne se pose que **pendant** une partition.
6. « **MongoDB est CP, Cassandra est AP.** » Simplification abusive. La plupart des bases modernes sont configurables, souvent **requête par requête**. Cassandra permet de choisir le niveau de cohérence de chaque lecture et de chaque écriture (**ONE**, **QUORUM**, **ALL**). La bonne question n'est pas « quelle base choisir ? » mais « quel réglage pour quelle requête ? ».
7. « **CAP décrit le compromis quotidien d'un système distribué.** » Faux, et c'est le malentendu le plus important. CAP ne dit rien du fonctionnement en régime nominal, c'est-à-dire de la quasi-totalité du temps.

5.5 PACELC

DÉFINITION

PACELC (Daniel Abadi, 2012). En cas de **Partition**, il faut arbitrer entre **Availability** et **Consistency**. Else — en fonctionnement normal — il faut arbitrer entre **Latency** et **Consistency**.

La seconde moitié est celle qui gouverne le quotidien. Garantir la cohérence forte impose d'attendre la confirmation d'un quorum de réplicas avant de valider une écriture, donc au minimum un aller-retour réseau, souvent plusieurs. En reprenant la hiérarchie des latences de la section 2.3, un aller-retour dans un centre de données coûte environ 500 μ s : la cohérence forte se paie donc en millisecondes, de façon parfaitement mesurable.

Les fournisseurs de bases gérées facturent d'ailleurs explicitement ce compromis. DynamoDB propose une lecture « éventuellement cohérente », rapide et moitié moins chère, et une lecture « fortement cohérente », plus lente et plus coûteuse.

Contexte	Exigence	Justification
Solde avant autorisation d'un retrait	Cohérence forte	Une valeur périmée autorise un découvert non couvert
Exécution d'un ordre de bourse	Cohérence forte	Conséquences juridiques et financières immédiates
Nombre de vues d'une œuvre	Cohérence éventuelle	Une approximation de quelques secondes est sans conséquence
Fil de recommandations	Cohérence éventuelle	L'indisponibilité coûte plus cher que l'imprécision

Le même système d'information combine presque toujours les deux régimes. Le travail de l'architecte consiste à savoir, pour chaque flux, dans quelle ligne il se range.

5.6 Localité des données

DÉFINITION

Principe de localité. Un programme pèse quelques kilo-octets ; un bloc de données pèse 128 mégaoctets. Il est des milliers de fois moins coûteux d'envoyer le programme vers la machine qui détient déjà le bloc que de transférer le bloc vers le programme.

Ce principe, formulé par MapReduce en 2004, structure toute l'architecture de Hadoop. Concrètement, le NameNode connaît la position de chaque bloc ; le planificateur demande à YARN un conteneur **sur le nœud qui détient déjà le bloc** ; en cas d'impossibilité, il se rabat sur un nœud du même rack, puis sur n'importe quel nœud.

Spark hérite du même mécanisme et expose quatre niveaux de localité, du meilleur au pire : **PROCESS_LOCAL** (les données sont dans la mémoire de l'executor), **NODE_LOCAL** (sur le même nœud), **RACK_LOCAL** (dans le même rack), **ANY** (n'importe où). La Spark UI affiche ce niveau pour chaque tâche ; un job majoritairement en **ANY** est un job qui transfère ses données par le réseau, donc un job lent. Vous l'observerez en TP5.

NUANCE

Dans les architectures cloud modernes, calcul et stockage sont **délibérément séparés** — le calcul s'exécute sur des instances élastiques, les données résident dans du stockage objet. La localité stricte disparaît. Elle est compensée par des réseaux à 25–100 Gb/s et par des caches locaux. Le principe reste vrai — minimiser les déplacements de données — mais son application a changé de forme : on ne rapproche plus le calcul du disque, on filtre et on projette au plus près de la source (*predicate pushdown*, module 2).

5.7 Le shuffle

À RETENIR

Le shuffle est le redécoupage des données à travers le réseau, rendu nécessaire lorsqu'une opération exige que des enregistrements situés sur des machines différentes se retrouvent ensemble. C'est le concept le plus important de tout ce cours.

Un **GROUP BY** doit rassembler toutes les lignes d'une même clé ; une jointure doit rapprocher les lignes de même clé issues de deux tables ; un tri global doit ordonner des éléments répartis partout. Dans les trois

cas, les données doivent traverser le réseau — et le shuffle cumule les trois opérations les plus lentes de la machine : écriture sur disque, transfert réseau, relecture sur disque.

Sans shuffle (transformations étroites)	Avec shuffle (transformations larges)
<code>filter</code> , <code>where</code>	<code>groupBy</code> , <code>groupByKey</code>
<code>map</code> , <code>select</code> , <code>withColumn</code>	<code>join</code> (hors diffusion)
<code>union</code>	<code>distinct</code>
<code>mapPartitions</code>	<code>orderBy</code> , <code>sortBy</code>
	<code>repartition</code> , <code>coalesce</code> (à la hausse)

Le shuffle est **irréductible** : on ne peut pas grouper sans rassembler. Tout l'art de l'optimisation distribuée consiste donc à le **réduire** — filtrer avant de joindre, diffuser la petite table plutôt que de shuffler les deux, pré-agréger localement — et non à le supprimer. C'est le programme entier du module 4.

6. Genèse et état de l'écosystème

6.1 Les trois articles de Google

Entre 2003 et 2006, Google publie trois articles qui décrivent l'infrastructure lui permettant d'indexer le web. Le code reste propriétaire ; les descriptions, elles, sont publiques — et c'est cette publication qui déclenche toute l'industrie du Big Data.

Année	Article	Contribution	Réimplémentation libre
2003	<i>The Google File System</i>	Système de fichiers distribué pour très gros fichiers sur matériel banal	HDFS
2004	<i>MapReduce</i>	Modèle de calcul distribué en deux phases, tolérant aux pannes	Hadoop MapReduce
2006	<i>BigTable</i>	Base de données distribuée orientée colonnes à très grande échelle	HBase

Les hypothèses de conception de GFS sont explicitement énoncées dans l'article, et elles expliquent l'intégralité du comportement de HDFS, y compris ses limites :

Hypothèse de GFS	Conséquence dans HDFS
Les pannes matérielles sont la norme, pas l'exception	Heartbeats, réplication, re-réplication automatique
Les fichiers sont très volumineux (plusieurs Gio)	Blocs de 128 Mio ; très mauvaise gestion des petits fichiers
Les lectures sont majoritairement séquentielles	Aucune optimisation de l'accès aléatoire ; pas d'index
Les écritures sont surtout des ajouts en fin de fichier	Impossibilité de modifier un fichier en place

Chaque « défaut » de HDFS est en réalité une conséquence assumée d'une hypothèse de conception. C'est l'une des questions de lecture du module.

6.2 De Nutch à Hadoop

En 2002, Doug Cutting et Mike Cafarella développent **Nutch**, un moteur de recherche web libre. Il fonctionne, mais ne dépasse pas quelques centaines de millions de pages. Les articles GFS et MapReduce leur apportent exactement la réponse manquante, et ils les réimplémentent au sein de Nutch.

En 2006, Cutting rejoint Yahoo!. La couche de stockage et de calcul est extraite de Nutch pour devenir un projet autonome, baptisé **Hadoop** d'après l'éléphant en peluche du fils de Cutting. Le projet devient projet de premier niveau de la fondation Apache en 2008, année où Yahoo! annonce faire tourner son index de production sur un cluster de 10 000 cœurs.

ATTENTION

Une erreur répandue attribue la création de Hadoop à Doug Cutting et **Tom White**. Tom White est l'auteur de *Hadoop: The Definitive Guide*, la référence documentaire du projet et un contributeur important — mais les créateurs sont **Doug Cutting** et **Mike Cafarella**.

6.3 Les trois âges de Hadoop

	Hadoop 1 (2011)	Hadoop 2 (2013)	Hadoop 3 (2017→)
Gestion des ressources	JobTracker monolithique	YARN	YARN + conteneurs opportunistes
Moteurs de calcul	MapReduce seul	MapReduce, Spark, Tez, Flink...	Idem, Spark dominant
Haute disponibilité	Aucune (NameNode = SPOF)	Deux NameNodes actif/veille	Plus de deux NameNodes
Tolérance au stockage	Réplication ×3 (+200 %)	Réplication ×3	Erasure coding (+50 %)
Passage à l'échelle	≈ 4 000 nœuds	≈ 10 000 nœuds	> 10 000 nœuds
Stockage objet	Non	Support S3 partiel	S3A mature, Azure, GCS

La rupture décisive est Hadoop 2 et l'introduction de YARN, qui sépare la gestion des ressources du modèle de calcul. C'est ce qui a permis à Spark de s'exécuter sur l'infrastructure Hadoop — et, paradoxalement, ce qui a condamné MapReduce.

6.4 Ce qui n'est plus à apprendre

Une large part des supports disponibles en ligne enseigne encore des technologies abandonnées. Savoir les identifier fait partie du métier.

Technologie	Rôle historique	État	À utiliser à la place
Apache Pig	Langage de flux de données	Dernière version 2017	Spark SQL, DataFrames
Apache Storm	Traitement de flux	Marginal	Flink, Structured Streaming
Apache Tez	Moteur DAG pour Hive	En déclin	Spark, Trino
Apache Oozie	Ordonnancement	Marginal	Airflow, Dagster, Prefect
Apache Flume	Collecte de journaux	Fin de vie	Kafka Connect, Fluent Bit, Vector
Apache Mesos	Gestion de cluster	Retiré à l'Attic (2021)	Kubernetes
Apache Drill	SQL sur données brutes	Marginal	Trino, DuckDB
Sandbox Hortonworks	Environnement de TP	Arrêté après la fusion Cloudera	Conteneurs Docker

À RETENIR

Quatre critères permettent de juger soi-même de la vitalité d'un projet : la date de la dernière version publiée ; l'activité du dépôt sur les douze derniers mois ; la présence du projet dans les offres des grands fournisseurs cloud ; et le nombre d'offres d'emploi qui le mentionnent.

6.5 Ce qui compte aujourd'hui

Couche	Technologies	Pourquoi
Stockage	S3, ADLS, GCS, MinIO, HDFS	Le stockage objet a largement remplacé HDFS hors on-premise
Format de fichier	Parquet (dominant), ORC, Avro	Stockage colonne et compression : le socle de tout
Format de table	Iceberg, Delta Lake, Hudi	ACID et versionnement au-dessus du stockage objet
Calcul par lots	Spark, Trino, DuckDB, Polars	Spark à grande échelle ; DuckDB et Polars en dessous
Flux	Kafka, Flink, Structured Streaming	Kafka est le standard de fait du transport d'événements
Orchestration	Airflow, Dagster, Kubernetes	Kubernetes a remplacé YARN dans le cloud
Catalogue	Hive Metastore, Unity, Polaris, Glue	Sans catalogue, un lac devient un marécage
Transformation	dbt, Spark SQL	SQL versionné, testé, documenté

6.6 Hadoop est-il mort ?

La question revient systématiquement, et elle mérite une réponse honnête : **non, mais Hadoop a changé de rôle.**

Ont décliné : *Hadoop MapReduce* comme moteur de calcul, remplacé par Spark — plus rapide et infiniment plus simple à écrire ; *HDFS dans le cloud*, remplacé par le stockage objet, moins cher et sans serveur à administrer ; et les *distributions commerciales* (HDP, MapR), emportées par la consolidation du marché.

Restent bien vivants : *HDFS et YARN en on-premise*, dans les banques, les télécoms et le secteur public, partout où les données ne peuvent pas sortir du système d'information ; *les formats et le catalogue* — Parquet, ORC, Hive Metastore sont nés dans cet écosystème et sont aujourd'hui universels ; et surtout *les concepts* — localité, réplication, tolérance aux pannes, shuffle — qui structurent aussi bien Spark et Flink que BigQuery ou Snowflake.

À RETENIR

C'est la raison pour laquelle ce cours enseigne Hadoop **comme cas d'étude** et les concepts **comme objectif**. Formulation utilisable en entretien : « Hadoop n'est plus une plateforme de calcul de premier choix, mais c'est le socle conceptuel — et une partie du socle logiciel — de tout ce qui l'a suivi. HDFS et YARN restent très présents en on-premise. »

7. Exercices d'auto-évaluation

Ces exercices ne sont pas notés. Ils vous permettent de vérifier votre compréhension avant le TD1 et l'examen. Les corrigés figurent en section 8.

Exercice 1 — Qualification

Pour chacun des systèmes suivants, dites s'il relève du Big Data au sens de la définition de la section 1.2, et justifiez par un calcul ou par une contrainte explicite.

1. Un cabinet de gestion conserve l'historique quotidien de clôture de 3 000 titres depuis 1990, et calcule chaque soir des indicateurs de volatilité sur 20 ans d'historique.
2. Une plateforme de streaming musical enregistre chaque écoute — 4 millions par jour — et recalcule ses recommandations toutes les nuits.
3. Une salle de marché doit détecter en moins de 50 ms si un ordre entrant présente un profil de manipulation de cours, en le comparant aux 30 dernières minutes d'activité.
4. Un musée numérise sa collection : 120 000 images en très haute définition, 40 Tio au total, consultées quelques centaines de fois par jour.

Exercice 2 — Estimation

Un job de traitement doit lire 8 Tio de journaux stockés sur HDFS, en extraire trois colonnes, et écrire 40 Gio de résultat agrégé.

1. Le cluster compte 20 nœuds, chacun doté de 6 disques lisant à 180 Mio/s. Quel est le débit de lecture agrégé théorique, et le temps de lecture minimal ?
2. Le job met en réalité 4 heures. Quel facteur d'écart avec votre estimation ? Citez trois causes plausibles.
3. Si les journaux étaient stockés en Parquet et que les trois colonnes ne représentaient que 5 % du volume, quel serait le nouveau temps de lecture minimal ?

Exercice 3 — Fiabilité

Un cluster de 400 nœuds, chacun de MTBF 4 ans, stocke 2 Pio en blocs de 128 Mio répliqués 3 fois.

1. Combien de pannes de nœud faut-il attendre par semaine ?
2. Combien de blocs distincts le cluster contient-il (avant réplication) ?
3. Si 8 nœuds tombent simultanément, quelle est la probabilité qu'un bloc donné soit perdu, en supposant un placement uniforme ? Combien de blocs perdus attend-on ?
4. En quoi le *rack awareness* modifie-t-il ce raisonnement ?

Exercice 4 — CAP et PACELC

Pour chacune des affirmations suivantes, dites si elle est vraie ou fausse et corrigez-la si nécessaire.

1. « Un système distribué doit choisir deux propriétés parmi C, A et P. »
2. « Cassandra est une base AP, elle ne peut donc pas offrir de cohérence forte. »
3. « CAP explique pourquoi une base distribuée est plus lente qu'une base locale. »
4. « Renoncer à la cohérence forte permet de réduire la latence même sans panne. »

8. Corrigés des exercices

Corrigé de l'exercice 1

1. **Non.** $3\,000 \text{ titres} \times 35 \text{ ans} \times 252 \text{ séances} \approx 26 \text{ millions de lignes}$, soit quelques gigaoctets. Un ordinateur portable avec pandas ou DuckDB traite cela en quelques secondes. Le distribuer serait une faute d'ingénierie.
2. **Cas limite, plutôt non.** 4 millions d'écoutes par jour, à environ 100 octets par événement, font 400 Mio par jour, soit 146 Gio par an. Le traitement nocturne n'impose aucune contrainte de latence. Un serveur à 256 Gio de RAM traite plusieurs années d'historique en mémoire. La distribution ne se justifierait qu'au-delà de quelques années cumulées, ou si le volume par événement était bien plus élevé.
3. **Oui**, mais pas pour une raison de volume — pour une raison de **latence sous charge**. La contrainte de 50 ms sur un flux continu, avec jointure sur un état glissant, impose une architecture de traitement de flux distribuée et partitionnée. C'est l'exemple type d'un problème Big Data à petit volume.
4. **Non.** 40 Tio, c'est beaucoup de stockage, mais quelques centaines de consultations par jour sur des fichiers indépendants ne posent aucun problème de calcul distribué. Un serveur de fichiers ou un stockage objet suffit. C'est un problème de capacité, pas de Big Data.

À RETENIR

Deux enseignements de cet exercice : un très gros volume n'implique pas le Big Data (cas 4), et un petit volume ne l'exclut pas (cas 3). Seul le rapport entre le traitement souhaité et la capacité d'une machine décide.

Corrigé de l'exercice 2

1. Débit agrégé = $20 \times 6 \times 180 = 21\,600 \text{ Mio/s} \approx 21,1 \text{ Gio/s}$. Temps minimal = $8 \times 1\,048\,576 / 21\,600 \approx 388 \text{ s}$, soit environ 6 min 30.
2. 4 heures contre 6 min 30 : un facteur d'environ **37**. Causes plausibles : (a) le format est ligne — CSV ou JSON — donc on lit 100 % du volume pour n'exploiter que trois colonnes, et le décodage texte consomme énormément de CPU ; (b) un shuffle massif lors de l'agrégation, avec écriture puis relecture sur disque ; (c) un déséquilibre de partitions, quelques tâches traitant l'essentiel du volume ; (d) un nombre de partitions mal choisi, provoquant soit de la contention, soit du *spill* mémoire ; (e) la lecture n'est pas locale et transite par le réseau.
3. 5 % de 8 Tio = 0,4 Tio. Temps minimal = $0,4 \times 1\,048\,576 / 21\,600 \approx 19 \text{ s}$. C'est l'intérêt du stockage colonne, mesuré au TP2 : on ne lit que les colonnes réellement demandées.

Corrigé de l'exercice 3

1. MTBF = 4 ans = 1 460 jours. Pannes par jour = $400 / 1\,460 \approx 0,274$. Par semaine : $0,274 \times 7 \approx 1,9 \text{ panne}$, soit environ deux par semaine.

2. $2 \text{ Pio} = 2 \times 1\,024^3 \text{ Mio} = 2\,147\,483\,648 \text{ Mio}$. Divisé par 128 Mio : **16 777 216 blocs**, soit environ 16,8 millions.
3. $P = C(8,3) / C(400,3) = 56 / 10\,586\,800 \approx 5,3 \times 10^{-6}$. Espérance de blocs perdus = $5,3 \times 10^{-6} \times 16\,777\,216 \approx 89 \text{ blocs}$. Le résultat est contre-intuitif : une probabilité minuscule par bloc devient une perte certaine dès lors que le nombre de blocs est très grand.
4. Le calcul suppose des pannes **indépendantes** et un placement **uniforme**. Or les pannes réelles sont corrélées : une alimentation ou un commutateur défaillant fait tomber un rack entier. Le *rack awareness* impose qu'au moins un réplica se trouve dans un autre rack ; la perte d'un rack complet devient alors survivable par construction, ce que le modèle uniforme ne capture pas.

Corrigé de l'exercice 4

1. **Faux**. P n'est pas un choix : les partitions réseau surviennent. Un système distribué doit toujours tolérer P. Le choix — C ou A — ne se pose que **pendant** une partition.
2. **Faux**. Cassandra permet de régler le niveau de cohérence par requête. Avec **QUORUM** en lecture et en écriture, elle offre une cohérence forte, au prix de la latence et de la disponibilité en cas de partition.
3. **Faux**. CAP ne dit rien du régime nominal. C'est PACELC qui formalise le compromis quotidien entre cohérence et latence.
4. **Vrai**. C'est exactement la seconde moitié de PACELC : **Else Latency or Consistency**. En dehors de toute panne, la cohérence forte exige des allers-retours de quorum qui coûtent des millisecondes.

9. Glossaire du module

Terme	Définition
Big Data	Traitement qui ne tient pas dans les ressources d'une machine unique et que l'on résout en le distribuant
Scale-up / scale-out	Passage à l'échelle vertical (machine plus puissante) / horizontal (davantage de machines)
Commodity hardware	Matériel de série, standard et interchangeable — et non matériel bon marché ou de récupération
MTBF	<i>Mean Time Between Failures</i> — temps moyen entre deux pannes d'un composant
Partitionnement (sharding)	Découpage d'un jeu de données en morceaux disjoints répartis sur plusieurs nœuds
Réplication	Duplication de chaque morceau en n exemplaires sur des nœuds distincts
Skew	Déséquilibre de la répartition : une partition bien plus grosse que les autres
Erasur coding	Codage correcteur offrant une tolérance équivalente à la réplication pour un surcoût d'espace bien moindre
CAP	En cas de partition réseau, il faut choisir entre cohérence et disponibilité
PACELC	Raffinement de CAP : sinon (Else), il faut choisir entre latence et cohérence
Cohérence éventuelle	Les réplicas convergent, sans garantie sur le délai
Localité des données	Placer le calcul sur le nœud qui détient déjà les données
Shuffle	Redécoupage des données à travers le réseau, imposé par un groupement, une jointure ou un tri
Lignage (lineage)	Suite des transformations ayant produit une donnée ; permet de la recalculer après une panne
Schema-on-write / on-read	Le schéma est imposé à l'écriture / interprété à la lecture
Data lake	Stockage de données brutes sans schéma imposé
Data swamp	Lac devenu inexploitable faute de catalogue et de gouvernance
Lakehouse	Lac augmenté d'une couche de table transactionnelle (Iceberg, Delta, Hudi)
Predicate pushdown	Application des filtres au plus près du stockage, pour ne lire que le nécessaire